RECHERCHE OPERATIONEL

MINI-PROJETS: ORDONNANCEMENT D'UN PIPELINE CI/CD (DEVOPS)

Intelligence Artificielle et Génie Informatique (IAGI)

membre du group :
ADAM Aderram
YASSINE Jahjah
ABDERRAHMANE Hajjam
SOUHAIL Kharbouch

encadré par Mr:
Kamouss Abdessamad

Année Universitaire 2025-2026

SOMMAIRE

01 Répartition des tâches dans l'équipe de projet

02 Une courte introduction

03 Phase 1 : Présentation du contexte du projet

04 Phase 2 : Modélisation

05 Phase 3 : Résolution

06 Phase 4 : Analyse Managériale

07 Conclusion et remerciment

RÉPARTITION DES TÂCHES DANS L'ÉQUIPE DE PROJET

Tâches	ADAM Aderram	YASSINE Jahjah	ABDERRAHMANE Hajjam	SOUHAIL Kharbouch
Chef du projet				
Analyse du projet				
Jeu des données				
Modélisation				
Implémentation Python				
Analyse des Résultats				
Rédaction du rapport				
Préparation slides				

INTRODUCTION

L'optimisation constitue un pilier fondamental de la Recherche Opérationnelle, en particulier pour l'analyse de systèmes complexes soumis à des contraintes de temps, de ressources et de dépendances. Elle permet de modéliser des problèmes réels de manière formelle et de déterminer des solutions efficaces visant à améliorer les performances globales d'un système.

Dans le cadre du module de Recherche Opérationnelle, ce projet a pour objectif d'appliquer les méthodes théoriques d'optimisation, telles que la programmation linéaire, la théorie des graphes et les algorithmes de flots, à une problématique concrète issue du domaine du numérique. Le choix s'est porté sur l'étude et l'optimisation d'un pipeline CI/CD, élément central des pratiques DevOps modernes.

Un pipeline CI/CD peut être représenté comme un ensemble de tâches interconnectées, caractérisées par des durées d'exécution et des contraintes de précédence. Cette structure se prête naturellement à une modélisation en Recherche Opérationnelle, notamment à travers les concepts d'ordonnancement et de graphes.
L'objectif de ce travail est d'analyser l'organisation de ces tâches et d'optimiser leur enchaînement afin de réduire le temps total d'exécution du pipeline, tout en respectant les contraintes imposées. Cette approche permet de mettre en évidence l'intérêt des techniques d'optimisation dans l'amélioration des processus de déploiement logiciel, en particulier dans des contextes critiques tels que les applications bancaires.

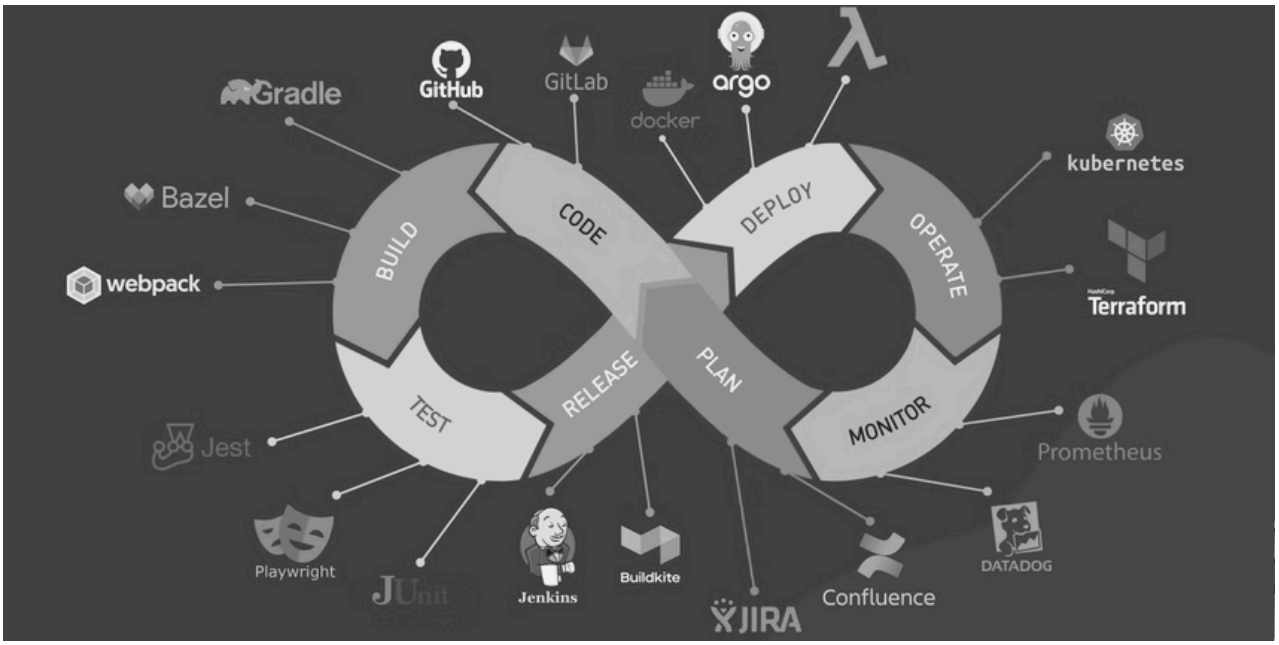
PRÉSENTATION DU CONTEXTE DU PROJET

Le déploiement d'une nouvelle version d'une application bancaire critique constitue une opération particulièrement sensible. Il doit garantir un niveau élevé de sécurité, de fiabilité et de disponibilité afin d'assurer la continuité des services financiers et de préserver la confiance des utilisateurs. Toute erreur ou interruption lors de cette phase peut engendrer des conséquences importantes, tant sur le plan technique qu'opérationnel.
Dans ce contexte, le déploiement ne peut pas être réalisé de manière manuelle ou non structurée. Il repose sur un pipeline DevOps strict, intégrant plusieurs étapes essentielles telles que :

- la compilation du code source ;
- l'exécution des tests unitaires ;
- la création d'images Docker ;
- les tests d'intégration ;
- les analyses de sécurité ;
- le déploiement vers les environnements de validation ou de production.

L'approche DevOps s'impose ainsi comme une solution efficace permettant l'automatisation, la traçabilité et la fiabilité du cycle de déploiement. Elle vise à rapprocher les équipes de développement et d'exploitation afin d'accélérer la livraison des applications tout en maintenant un haut niveau de qualité et de sécurité.

LE CYCLE DEVOPS ET LE PIPELINE CI/CD

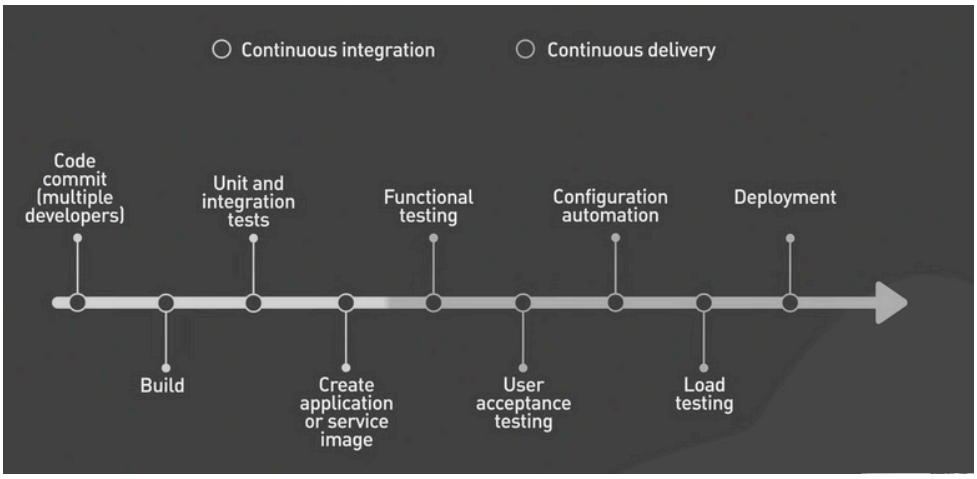


Le cycle DevOps repose sur une succession continue d'étapes interconnectées : Plan, Code, Build, Test, Release, Deploy, Operate et Monitor. Ce cycle permet d'assurer une amélioration continue du logiciel, depuis sa conception jusqu'à son exploitation en production.
La figure ci-dessus illustre ce cycle DevOps et met en évidence :

- les principaux composants du cycle ;
- des exemples d'outils utilisés à chaque étape ;
- la continuité du processus, du développement jusqu'à la supervision en production ;
- la décomposition d'un pipeline CI/CD en un ensemble de tâches dépendantes les unes des autres.

Cette décomposition rend possible l'analyse du pipeline comme un problème d'ordonnancement, ce qui constitue le cœur de ce projet en Recherche Opérationnelle.

La figure ci-contre illustre de manière claire et structurée le pipeline de développement logiciel moderne, en mettant en évidence les étapes clés de l'intégration continue (CI) et de la livraison continue (CD).



- Intégration continue (CI) : Elle commence par les commits de code des développeurs, suivis de la compilation, des tests unitaires et d'intégration, puis de la création d'une image de l'application ou du service.
- Livraison continue (CD) : Elle enchaîne avec les tests fonctionnels, l'acceptation utilisateur, l'automatisation de la configuration, les tests de charge, et enfin le déploiement.

PRÉSENTATION DU JEU DE DONNÉES

Le jeu de données utilisé dans ce projet représente un pipeline CI/CD d'une application bancaire. Il est composé d'un ensemble de tâches correspondant aux différentes étapes du déploiement.
Chaque tâche est caractérisée par :

- un identifiant ;
- une durée d'exécution, exprimée en minutes ;
- une liste de prédécesseurs, traduisant les contraintes de précédence entre les tâches.

Le tableau suivant présente l'ensemble des tâches du pipeline :

Code	Tâche	Durée (min)	Prédécesseurs
A	Git Checkout	2	-
B	Compile Backend	15	A
C	Compile Frontend	10	A
D	Unit Tests Back	8	B
E	Unit Tests Front	5	C
F	Build Docker Image	12	B, C
G	Security Scan (SAST)	20	A
H	Integration Tests	25	D, E, F
I	Deploy to Prod	10	G, H

Ce jeu de données permet de représenter un pipeline réaliste, combinant des tâches séquentielles et parallélisables, et constitue une base pertinente pour l'étude de son ordonnancement.

ENJEUX ET GAINS ATTENDUS

Le principal enjeu de ce projet est la réduction du Time-to-Market, c'est-à-dire le temps total nécessaire pour déployer une nouvelle version de l'application bancaire.

Un ordonnancement inefficace peut entraîner :

- des temps d'attente inutiles ;
- une sous-utilisation des ressources disponibles ;
- des retards dans la mise en production.

À l'inverse, un ordonnancement optimisé du pipeline CI/CD permet :

- une exécution plus rapide du processus de déploiement ;
- une meilleure exploitation des tâches parallélisables ;
- une identification claire des tâches critiques ;
- une amélioration globale de la fiabilité et de la performance du pipeline.

MODÉLISATION DE PROBLÈME

INTRODUCTION DE LA PARTIE MODÉLISATION

Dans le cadre du déploiement d'applications informatiques critiques, notamment dans un environnement bancaire, la maîtrise du temps de mise en production constitue un enjeu majeur. Le pipeline CI/CD (Continuous Integration / Continuous Deployment) regroupe un ensemble de tâches techniques telles que la compilation, les tests, les analyses de sécurité et le déploiement final. Ces tâches sont soumises à des contraintes de précédence : certaines peuvent être exécutées en parallèle, tandis que d'autres doivent impérativement attendre l'achèvement d'étapes antérieures.

La modélisation de ce pipeline s'inscrit dans le domaine de la Recherche Opérationnelle, et plus précisément dans les méthodes d'ordonnancement de projets. Afin d'analyser la durée totale du déploiement et d'identifier les tâches critiques influençant directement le Time-to-Market, nous adoptons une approche basée sur les méthodes PERT (Program Evaluation and Review Technique) et CPM (Critical Path Method).

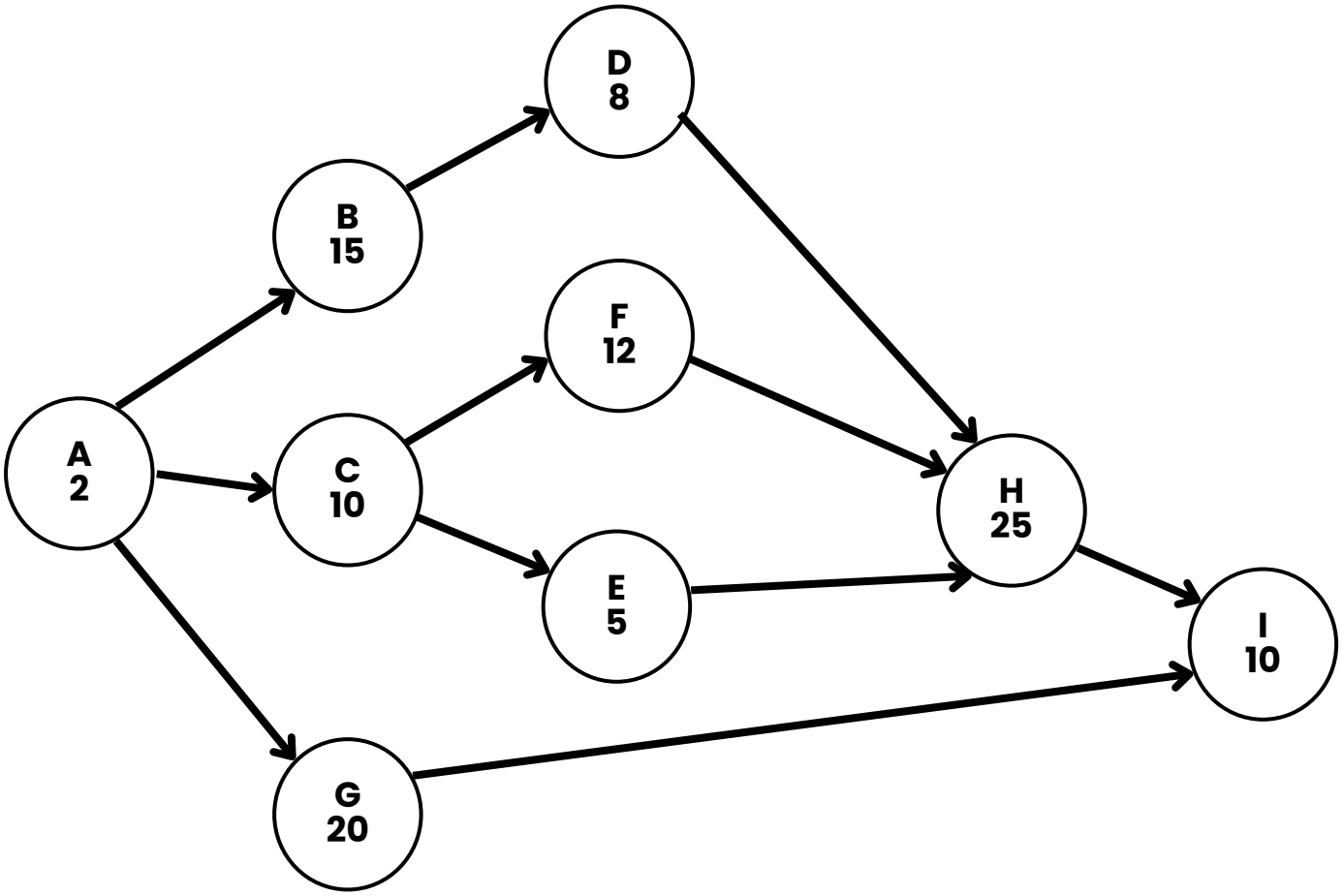
Cette modélisation permet de représenter formellement les différentes tâches du pipeline CI/CD, leurs durées ainsi que leurs relations de dépendance. Elle offre ainsi un cadre analytique pour calculer les dates de début et de fin au plus tôt et au plus tard, et pour déterminer le chemin critique, c'est-à-dire la séquence de tâches dont tout retard entraîne automatiquement un retard du déploiement global

CONTEXTE ET CAHIER DES CHARGES

Le déploiement dune nouvelle version dune application bancaire critique suit un pipeline DevOps strict : Tests unitaires, Build Docker, Tests dintégration, Scan de sécurité (SAST), etc. L'enjeu est de minimiser la durée totale (Time-to-Market) tout en respectant les dépendances techniques.

MODÉLISATION GRAPHIQUE (PERT)

Le graphe de modélisation PERT est un outil de la Recherche Opérationnelle permettant de représenter l'ordonnancement des tâches d'un projet complexe.
Il met en évidence les relations de dépendance entre les différentes activités ainsi que leurs durées respectives.
Chaque tâche est intégrée dans un réseau orienté traduisant les contraintes de précédence du projet.
Cette représentation facilite l'analyse de l'exécution parallèle ou séquentielle des tâches.
Le graphe PERT permet de calculer les dates de début et de fin au plus tôt et au plus tard.
Il aide également à identifier le chemin critique du projet.
Ainsi, il constitue un support essentiel pour optimiser la durée totale du déploiement



MODÉLISATION MATHÉMATIQUE (PL)

Pour automatiser la résolution, nous formulons le problème sous forme canonique de programmation linéaire.
Cette formulation permet de représenter les contraintes et la fonction objectif de manière mathématique claire.
Elle facilite l'utilisation des algorithmes standards d'optimisation pour obtenir une solution optimale.
Ainsi, le problème peut être résolu efficacement à l'aide d'outils informatiques adaptés.

VARIABLES ET FONCTION OBJECTIF

Soit xi la date de début au plus tôt de la tâche i . Soit Z la date de fin.

MinimiserZ

$$\begin{cases} x_A \geq 0 & \\ x_B \geq x_A + 2, \quad x_C \geq x_A + 2, \quad x_G \geq x_A + 2 & \text{(Après A)} \\ x_D \geq x_B + 15, \quad x_F \geq x_B + 15 & \text{(Après B)} \\ x_E \geq x_C + 10, \quad x_F \geq x_C + 10 & \text{(Après C)} \\ x_H \geq x_D + 8, \quad x_H \geq x_E + 5, \quad x_H \geq x_F + 12 & \text{(Convergence vers H)} \\ x_I \geq x_G + 20, \quad x_I \geq x_H + 25 & \text{(Convergence vers I)} \\ Z \geq x_I + 10 & \text{(Fin Projet)} \end{cases}$$

RÉSOLUTION DE PROBLÈME

INTRODUCTION

La résolution du problème d'ordonnancement repose sur l'exploitation du graphe de modélisation PERT établi précédemment. À partir des durées des tâches et des contraintes de précédence, il s'agit de calculer les dates de début et de fin au plus tôt et au plus tard pour chaque activité. Cette étape permet de déterminer la durée totale du projet et d'identifier le chemin critique, dont toute perturbation entraîne un retard global du déploiement. La résolution constitue ainsi une phase essentielle pour évaluer la performance du pipeline CI/CD et optimiser le Time-to-Market.

TRI TOPOLOGIE : (ALGORITHME DE KAHN)

Le tri topologique est une méthode utilisée pour ordonner les tâches d'un projet en respectant les contraintes de précédence.
Il s'applique aux graphes orientés sans cycle, comme le graphe PERT.
Cette technique permet de déterminer un ordre d'exécution valide des différentes activités.
Elle facilite le calcul des dates au plus tôt et au plus tard.
Le tri topologique constitue ainsi une étape clé dans la résolution du problème d'ordonnancement

PSEUDO CODE :

```
Q ← ENSEMBLE DES TÂCHES AVEC IN_DEGREE = 0
ORDRE ← LISTE VIDE
TANT QUE Q N'EST PAS VIDE
  U ← EXTRAIRE(Q)
  AJOUTER U À ORDRE
  POUR CHAQUE SUCCESSEUR V DE U
    IN_DEGREE[V] ← IN_DEGREE[V] − 1
    SI IN_DEGREE[V] = 0
      AJOUTER V À Q
RETOURNER ORDRE
```

RESULTAT :

['A', 'B', 'C', 'G', 'D', 'E', 'F', 'H', 'I']

```python
def topological_sort(graph, in_degree):
    """
    Effectue le tri topologique en utilisant l'algorithme de Kahn.
    Assure que les tâches sont ordonnées en respectant les dépendances.
    """

    # Commencer avec les tâches qui n'ont pas de prédécesseurs
    queue = [t for t in in_degree if in_degree[t] == 0]

    topo_order = []

    while queue:
        # Retirer la première tâche de la file
        current = queue.pop(0)
        topo_order.append(current)

        # Réduire le degré entrant des successeurs
        for successor in graph[current]:
            in_degree[successor] -= 1

            # S'il ne reste aucun prédécesseur, ajouter à la file d'attente
            if in_degree[successor] == 0:
                queue.append(successor)

    return topo_order
```

DATE AU PLUS TOT

Le calcul des dates au plus tôt, appelé passage en avant, vise à déterminer la date de début et la date de fin les plus précoces possibles pour chaque tâche. Une tâche ne peut commencer que lorsque tous ses prédécesseurs sont terminés. Cette étape permet d'estimer la durée minimale du projet et repose sur l'ordre obtenu par le tri topologique.

PSEUDO CODE :

POUR CHAQUE TÂCHE T

  ES[T] ← 0

POUR CHAQUE TÂCHE T DANS L'ORDRE TOPOLOGIQUE

  EF[T] ← ES[T] + DURÉE[T]

  POUR CHAQUE SUCCESSEUR S DE T

    ES[S] ← MAX(ES[S], EF[T])

```python
def compute_earliest(tasks, graph, topo):
    """
    Calcule les temps de début au plus tôt (ES) et de fin au plus tôt (EF)
    """

    # Initialiser les temps de début au plus tôt à zéro
    earliest_start = {t: 0 for t in tasks}
    earliest_finish = {}

    for task in topo:
        # EF = ES + durée
        earliest_finish[task] = (
            earliest_start[task] + tasks[task]["duration"]
        )

        # Mettre à jour les temps de début au plus tôt des successeurs
        for successor in graph[task]:
            if earliest_start[successor] < earliest_finish[task]:
                earliest_start[successor] = earliest_finish[task]

    return earliest_start, earliest_finish
```

RESULTAT :

Code	Tâche	Durée (min)	ES (Début au plus tôt)	EF (Fin au plus tôt)
A	Git Checkout	2	0	2
B	Compile Backend	15	2	17
C	Compile Frontend	10	2	12
D	Unit Tests Back	8	17	25
E	Unit Tests Front	5	12	17
F	Build Docker Image	12	17	29
G	Security Scan (SAST)	20	2	22
H	Integration Tests	25	29	54
I	Deploy to Prod	10	54	64

DATE AU PLUS TARD

Le calcul des dates au plus tard, ou passage en arrière, consiste à déterminer les dates limites auxquelles chaque tâche peut débuter ou se terminer sans retarder l'ensemble du projet. Cette étape est essentielle pour analyser la flexibilité des tâches et s'effectue en parcourant le graphe dans l'ordre inverse du tri topologique.

PSEUDO CODE :

POUR CHAQUE TÂCHE T

LF[T] ← DURÉE_TOTALE_DU_PROJET

POUR CHAQUE TÂCHE T DANS L'ORDRE TOPOLOGIQUE INVERSE

LS[T] ← LF[T] – DURÉE[T]

POUR CHAQUE PRÉDÉCESSEUR P DE T

LF[P] ← MIN(LF[P], LS[T])

```python
def compute_latest(tasks, topo, project_duration):
    """
    Calcule les temps de début au plus tard (LS) et de fin au plus tard (LF)
    """

    # Initialiser les temps de fin au plus tard à la durée du projet
    latest_finish = {t: project_duration for t in tasks}
    latest_start = {}

    # Parcourir les tâches dans l'ordre topologique inverse
    for task in reversed(topo):
        # LS = LF - durée
        latest_start[task] = (
            latest_finish[task] - tasks[task]["duration"]
        )

        # Mettre à jour les temps de fin au plus tard des prédécesseurs
        for predecessor in tasks[task]["pred"]:
            if latest_finish[predecessor] > latest_start[task]:
                latest_finish[predecessor] = latest_start[task]

    return latest_start, latest_finish
```

RESULTAT :

Code	Tâche	Durée (min)	LS (Début au plus tard)	LF (Fin au plus tard)
I	Deploy to Prod	10	54	64
H	Integration Tests	25	29	54
G	Security Scan (SAST)	20	34	54
F	Build Docker Image	12	17	29
D	Unit Tests Back	8	21	29
E	Unit Tests Front	5	24	29
B	Compile Backend	15	2	17
C	Compile Frontend	10	7	17
A	Git Checkout	2	0	2

LA MARGE :

La marge, ou slack, représente le retard maximal qu'une tâche peut subir sans affecter la durée totale du projet. Le calcul des marges permet d'identifier les tâches flexibles et celles nécessitant une attention particulière. Une marge nulle indique une tâche critique.

PSEUDO CODE :

POUR CHAQUE TÂCHE T

SLACK[T] ← LS[T] – ES[T]

```python
def compute_slack(tasks, earliest_start, latest_start):
    """
    Calcule la marge pour chaque tâche :
    Marge = LS - ES
    """

    slack = {}
    for task in tasks:
        slack[task] = latest_start[task] - earliest_start[task]

    return slack
```

IDENTIFICATION DE CHEMIN CRITIQUE

Le chemin critique correspond à la séquence de tâches dont la marge est nulle. Ces tâches déterminent directement la durée totale du projet, et tout retard sur l'une d'elles entraîne un retard global. L'identification du chemin critique est essentielle pour le pilotage et le contrôle du projet

PSEUDO CODE :

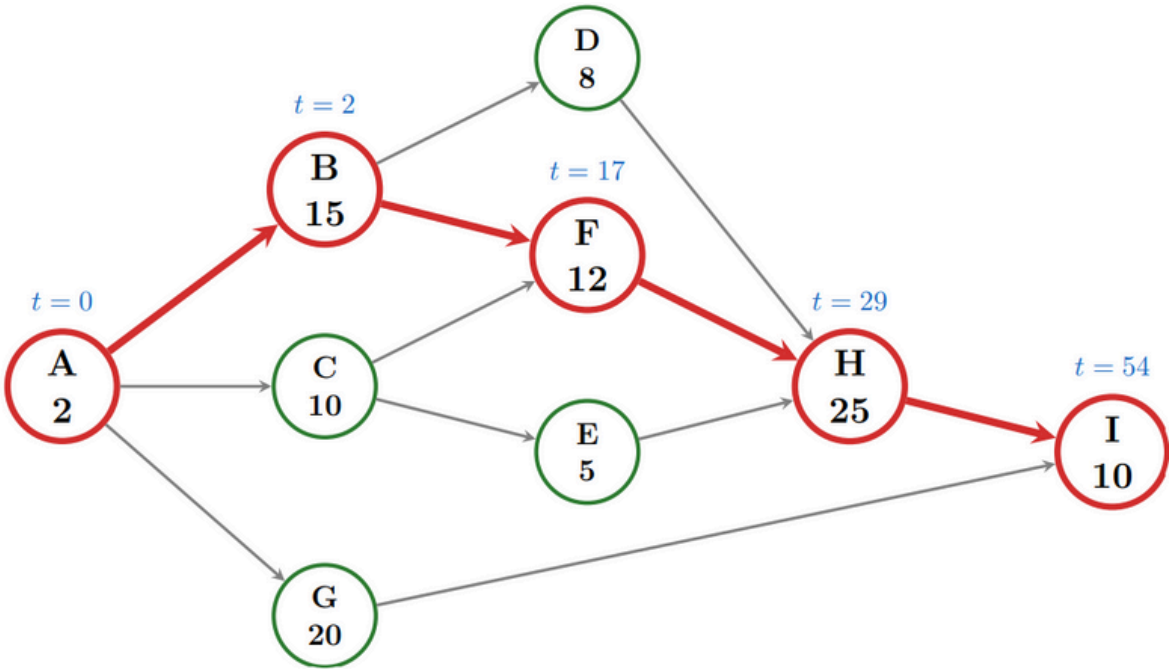
CHEMIN_CRITIQUE ← LISTE VIDE
POUR CHAQUE TÂCHE T DANS L'ORDRE TOPOLOGIQUE
  SI SLACK[T] = 0
    AJOUTER T À CHEMIN_CRITIQUE

```python
def critical_path(topo, slack):
    """
    Extrait le chemin critique :
    tâches avec une marge nulle
    """

    return [task for task in topo if slack[task] == 0]
```

RESULTAT :

Le diagramme suivant représente le graphe des potentiels-tâches. Le chemin critique est mis en évidence en rouge. Il détermine la durée incompressible du projet.



CONCLUSION

L'ensemble des algorithmes présentés constitue une approche complète et cohérente pour la résolution du problème d'ordonnancement basé sur la modélisation PERT. Le tri topologique permet d'établir un ordre d'exécution valide des tâches en respectant les contraintes de précédence. Le calcul des dates au plus tôt et au plus tard offre une vision temporelle précise du projet, tandis que le calcul des marges met en évidence la flexibilité de chaque tâche. Enfin, l'identification du chemin critique permet de déterminer les activités dont la maîtrise est essentielle, car tout retard sur celles-ci impacte directement la durée totale du projet. Ainsi, ces algorithmes constituent des outils fondamentaux pour l'analyse, la planification et l'optimisation des projets complexes

ANALYSE MANAGÉRIALE DES RESULTATS

L'analyse managériale des résultats du projet PERT a pour objectif de fournir des informations pratiques pour la prise de décision et la gestion du projet. Contrairement à la partie modélisation ou programmation, cette section ne s'intéresse pas aux détails techniques de l'algorithme, mais aux implications managériales des résultats calculés :

- Durée totale du projet : connaître le temps minimal nécessaire pour atteindre l'objectif.
- Chemin critique : identifier les tâches dont le retard entraîne un décalage global.
- Marges des tâches : évaluer la flexibilité disponible pour les tâches non critiques.

Cette analyse permet aux managers de prioriser les ressources, anticiper les risques et optimiser la planification, garantissant ainsi la réussite du projet dans les délais impartis.

1. Durée totale du projet

- Durée minimale du projet : 64 minutes
- La durée est déterminée par le chemin critique : A → B → F → H → I
- Cette durée représente le temps obligatoire pour terminer le projet, sans aucune marge sur ces tâches critiques.

Implications managériales :

1. Les tâches critiques doivent être suivies de près pour éviter tout retard global.
2. Toute modification ou retard sur ces tâches impacte directement la date de fin du projet.
3. La planification et l'allocation de ressources doivent se concentrer sur ces tâches.

2. Chemin critique

Le chemin critique est :

A → B → F → H → I

- Marge = 0 pour chaque tâche du chemin critique.
- Tâches critiques : toute perturbation entraîne un retard du projet.

Recommandations managériales :

- Prioriser les ressources sur A, B, F, H et I.
- Suivi quotidien pour détecter et corriger tout retard.
- Prévoir ressources supplémentaires pour sécuriser ces tâches.

3. Marges des autres tâches

Tâche	ES (Start au plus tôt)	EF (Finish au plus tôt)	LS (Start au plus tard)	LF (Finish au plus tard)	Marge
C	2	12	7	17	5
G	2	22	34	54	32
D	17	25	21	29	4
E	12	17	24	29	12

- Les tâches hors chemin critique disposent de marges positives, offrant une certaine flexibilité.
- Par exemple, la tâche G peut commencer jusqu'à 34 minutes plus tard sans affecter le projet.

Implications managériales :

1. Flexibilité dans la planification : ces tâches peuvent être retardées ou réaffectées si nécessaire.
2. Optimisation des ressources : rediriger temporairement le personnel vers les tâches critiques.
3. Gestion des imprévus : les marges servent de tampon pour absorber de petits retards.

4. Analyse des risques

1. Haute criticité : A, B, F, H, I – toute perturbation entraîne un retard immédiat.
2. Moyenne criticité : D, E, C – petites marges, nécessitent une surveillance mais moins urgente.
3. Basse criticité : G – grande marge, faible risque de retarder le projet.

Recommandations :

- Focus sur les tâches critiques pour éviter les dépassements.
- Exploiter les marges pour rééquilibrer les ressources en cas de besoin.
- Préparer un plan de contingence pour les tâches critiques.

5. Impacts managériaux globaux

- Priorisation claire : le manager sait exactement quelles tâches sont critiques.
- Optimisation des ressources : utiliser les marges pour améliorer l'efficacité globale.
- Réduction des risques de retard : la combinaison chemin critique + marges permet une planification proactive.
- Prise de décision facilitée : savoir quelles tâches surveiller ou décaler pour maximiser la performance du projet.

6. Conclusion

Avec PERT :

- La durée minimale du projet est clairement identifiée (64 minutes).
- Le chemin critique permet de savoir où concentrer les efforts.
- Les tâches avec marge offrent de la flexibilité pour gérer les ressources et les imprévus.

=> Le projet peut être piloté efficacement, avec un contrôle strict des tâches critiques et une utilisation optimale des marges pour maximiser la réussite du projet.

CONCLUSION

Ce projet a permis de démontrer concrètement l'apport fondamental de la Recherche Opérationnelle dans l'analyse et l'optimisation de systèmes complexes soumis à des contraintes temporelles et organisationnelles. À travers l'étude d'un pipeline CI/CD d'une application bancaire critique, nous avons montré qu'un processus de déploiement logiciel peut être modélisé de manière rigoureuse comme un problème d'ordonnancement, intégrant des tâches interdépendantes, des durées d'exécution et des contraintes de précédence.

L'utilisation des méthodes PERT/CPM, combinée aux algorithmes de tri topologique et de calcul des dates au plus tôt et au plus tard, a permis d'identifier avec précision la durée minimale du projet ainsi que le chemin critique déterminant le Time-to-Market. Cette approche analytique met en évidence les tâches nécessitant une attention particulière, tout en révélant les marges de flexibilité exploitables pour une meilleure allocation des ressources.

Au-delà de l'aspect technique, l'analyse managériale des résultats souligne l'intérêt opérationnel de ces méthodes pour la prise de décision. Elle offre aux responsables de projet une vision claire des priorités, des risques potentiels et des leviers d'optimisation, contribuant ainsi à une gestion proactive et efficace du déploiement. Dans un contexte bancaire où la fiabilité, la sécurité et la rapidité sont essentielles, l'optimisation du pipeline CI/CD constitue un enjeu stratégique majeur.

En conclusion, ce travail illustre parfaitement comment les outils de la Recherche Opérationnelle peuvent être appliqués à des problématiques réelles du domaine numérique et DevOps. Il met en lumière leur valeur ajoutée tant sur le plan technique que managérial, et ouvre la voie à des perspectives d'amélioration futures, telles que l'intégration de ressources limitées, l'incertitude sur les durées ou encore l'automatisation avancée de la prise de décision dans les pipelines de déploiement logiciel.

REMERCIEMENTS

Nous tenons à exprimer notre sincère gratitude à notre enseignant du module de Recherche Opérationnelle monsieur Kamouss Abdessamad pour son encadrement, ses orientations pédagogiques et la qualité de ses explications. Ses conseils avisés nous ont permis de mieux comprendre les concepts théoriques et de les appliquer à la réalisation de ce projet.

membre de groupe :
ADAM Aderram
YASSINE Jahjah
ABDERRAHMANE Hajjam
SOUHAIL Kharbouch